Helix Thready — helix_shims Interface Findings (+ SwiftUI Decision Memo)

Table of Contents

Rev	Date	Author	Change
1	2026-07-22	swarm (design · platforms)	First actual inspection of `vasic-digital/helix_shims` (local checkout at remote HEAD): full interface inventory, verified absence of ArkTS/Qt shims and of any design-token surface — closes the *inspection* half of `[OPEN: THREADY-DES-LIB-03]` / `[GAP: 8.5]`; adds the `THREADY-DES-LIB-02` SwiftUI decision memo (§6, decision pending)
2	2026-07-22	swarm (design · decisions)	§6 operator ruling recorded: **Option B adopted** (Compose-Multiplatform-only + thin SwiftUI host shell) — `THREADY-DES-LIB-02` **CLOSED (decided)**; §5 restated as the items-status section: `THREADY-DES-LIB-03` noted **narrowed** (inspection half delivered here; remainder = the actual ArkTS/Aurora client platform decision)

Scope guard. This file records what `helix_shims` *actually contains* as of the inspected commit, strictly read-only — no shim code was modified. It closes the **inspection** half of `[OPEN: THREADY-DES-LIB-03]` (“the `helix_shims` interface for ArkTS/Qt was not inspected `[GAP: 8.5]`”). The *re-verification of the AR/QT matrix cells once shim contracts exist* remains open — those contracts do not exist yet (§4). Section 6 is the decision memo for `THREADY-DES-LIB-02` (iOS host decision); as of rev 2 it carries the recorded operator ruling (**Option B adopted**) and the item is closed.

Table of Contents

1. What was inspected

- Repository: `vasic-digital/helix_shims` (private; GitHub default branch `main`) `[VERIFIED — gh api repos/vasic-digital/helix_shims, 2026-07-22]`.
- Local working copy: `/home/milos/Factory/projects/tools_and_research/helix_vpn/submodules/helix_shims` (a `helix_vpn` submodule), clean tree, at commit `a33e902f6c2ae544832fb0cfa815c45a242e7b33` (2026-07-07) — **identical to the remote `main` HEAD** at inspection time, so the findings below describe the repo's current published state `[VERIFIED — git log/status vs gh api …/commits/main]`.
- Method: full file-tree walk (build artifacts under `target/` excluded) + source read of every interface-bearing file; grep sweep for any design-token/theming/ArkTS/QML content.

2. What exists — verified interface inventory

`helix_shims` self-describes as "Per-platform tunnel shims: Apple `NEPacketTunnelProvider`, Android `VpnService`, Windows `wireguard-nt`, Linux `tun`, HarmonyOS Network Kit, Aurora OS Qt/tun", with MVP status "**Scaffolding only**" `[VERIFIED — README.md]`. What is actually implemented at HEAD:

2.1 Rust FFI core — `crates/helix-ffi` (the Dart/flutter_rust_bridge boundary)

`[VERIFIED — crates/helix-ffi/src/api.rs, 202 lines; Cargo.toml]`

- flutter_rust_bridge **v2, pinned 2.12.0**; `api.rs` is "the single hand-authored FFI surface" (its own words), Phase-0 **G5** minimal surface:
  - `pub struct ClientConfig { map_path: String, transport: String, … }`
  - `pub async fn start(cfg: ClientConfig) -> anyhow::Result<()>` (api.rs:90)
  - `pub async fn stop() -> anyhow::Result<()>` (api.rs:161)
  - `pub fn status_stream(sink: StreamSink<TunnelStatus>)` (api.rs:189)
  - `pub enum TunnelStatus { Connecting, Handshaking, Connected { transport, rtt_ms }, Reconnecting, Down { reason } }` (api.rs:67–73)
- Supporting modules: `fanout.rs` (465 ln, status fan-out), `project.rs` (298 ln), `frb_stub.rs` (96 ln), `runtime.rs`, `state.rs`; integration tests `tests/g5_ffi_boundary.rs`, `tests/g5_ui_contract.rs` `[VERIFIED — file tree + wc -l]`.

2.2 Android — `android/` (Kotlin `VpnService` + JNI)

`[VERIFIED — android/app/src/main/kotlin/dev/helixvpn/android/**, android/rust-jni/src/lib.rs]`

- Kotlin app: `HelixVpnService.kt`, `HelixTunnelConfig.kt`, `PlatformTunnelEvent.kt`, `MainActivity.kt`, unit + instrumented tests.

- JNI boundary `core/HelixNative.kt` (87 ln), backed by `android/rust-jni` (`libhelix_jni.so`):
  - `external fun nativeStart(mapPath: String, transport: String): Int`
  - `external fun nativeStop(): Int`
  - `external fun nativeSubscribeStatus(sink: TunnelStatusSink): Long`
  - `external fun nativeUnsubscribeStatus(handle: Long)`
  - callback `TunnelStatusSink.onStatus(kind, transport, rttMs, reason)`
- Cross-compile + JNI-signature-verification scripts under `android/scripts/`.

2.3 Apple — `apple/` (Rust C-ABI + Swift spike)

`[VERIFIED — apple/helix-ios-ffi/src/ffi.rs, 260 ln; apple/ios-spike/HelixTunnel/PacketTunnelProvider.swift, 311 ln]`

- `helix-ios-ffi`: plain C ABI (`#[no_mangle] extern "C"`), cbindgen-generated header (`apple/ios-spike/helix_core.h`): `helix_core_start(config)`, `helix_core_stop()`, `helix_core_tun_out(data, len)`, `helix_core_set_inbound_callback(cb, user_data)`, `helix_core_poll_status(buf, len)`, `helix_core_last_error(buf, len)`.
- `ios-spike/HelixTunnel/PacketTunnelProvider.swift`: a NetworkExtension `NEPacketTunnelProvider` host shell driving that C ABI — the in-house precedent for "thin Swift host over shared core" (relevant to §6).

3. What does not exist — verified absences

All of the following are **absent at HEAD** — verified by full directory walk (top-level dirs are exactly: `android/`, `apple/`, `crates/`, `upstreams/` + docs/config files):

- **No `harmonyos/` directory, no ArkTS code** — zero `.ets` files anywhere. The only HarmonyOS/ArkTS references are prose: the README's *planned* shim structure (`harmonyos/ # Network Kit ability`) and `CLAUDE.md:59` naming "ArkTS/C++ for HarmonyOS" as a planned language `[VERIFIED — grep sweep]`.
- **No `aurora/` directory, no Qt/QML/C++ shim code** — zero `.qml`/Qt files; "C++/QML for Aurora" appears only as the same planned-language prose `[VERIFIED — grep sweep]`.
- **No `windows/`, no `linux/` shim directories** — README: "Platform shims for other operating systems are future work" `[VERIFIED — README.md + tree]`.
- **No design-token, theming, or color surface of any kind** — a repo-wide grep for `design token | --accent | theme | color` over `*.rs`, `*.kt`, `*.swift`, `*.md` (build output excluded) returns only the CLAUDE.md planned-language line. The FFI vocabulary is exclusively tunnel lifecycle (`start/stop/status`), never UI `[VERIFIED — grep sweep, 2026-07-22]`.

4. Consequence for design-token consumption and the AR/QT columns

1. **`helix_shims` is a tunnel-lifecycle FFI layer, not a UI layer.** Nothing in its existing or planned interface carries visual state; design tokens will never flow *through* it. The platform-map's ground-truth note that ArkTS/Qt are "only reachable via native clients + `helix_shims`" (platform-map §2) should be read as: `helix_shims` supplies the *network* half of those native clients; the *UI* half — where tokens are consumed — is separate client code that does not exist yet anywhere in this repo `[VERIFIED — §2/§3 above]`.
2. **The token path for ArkTS/Qt is therefore independent of the shim contracts.** The machine-generated ArkTS binding already shipped by Thready (tokens-bridge/generated/arkts/thready_tokens.ets, tokens-bridge/README.md) remains a *contract-only* artifact awaiting a native ArkTS client, exactly as labeled there — this inspection confirms no such client (or shim half) exists to consume it yet.
3. **The AR/QT matrix columns stay plan-only.** platform-map §3 marks every AR/QT cell ASSUMED/ground-truth; nothing found here upgrades any cell. The re-verification half of `THREADY-DES-LIB-03` stays open until `harmonyos/` / `aurora/` shim contracts and their client scaffolds exist.

5. Status of the workable items (THREADY-DES-LIB-02 / -03)

THREADY-DES-LIB-02 — CLOSED (decided)

- **CLOSED (decided) by operator ruling, 2026-07-22** — **Option B adopted** (§6): the iOS host is Compose-Multiplatform-only with a thin SwiftUI host shell, per the memo's recommendation and the platform-map §2.1 sanctioned path. iOS idiom deviations are accepted as documented (§6, Option B risks). No dedicated SwiftUI component package will be built; the generated `ThreadyTokens.swift` contract stays on the shelf as labeled.
- **Remaining engineering dependency (not part of this item):** the `UI-Components-KMP` Yole→Thready retheme, which consumes the committed tokens-bridge `ThreadyColors.kt` contract — the Compose analogue of the executed React re-bind (react-token-rebind.md).

THREADY-DES-LIB-03 — NARROWED

- **Inspection half: delivered by this file** — the interface *was* inspected (2026-07-22, commit `a33e902f…`), with the concrete inventory in §2 and verified absences in §3. `[GAP: 8.5]`'s "uninspected" qualifier no longer holds.
- **What remains is narrower than the original wording:** since `helix_shims` was proven to contain **no ArkTS/Aurora surface at all** (§3) — and by design never will carry UI/token state (§4.1) — the remaining substance of the item is the **actual ArkTS/Aurora client platform decision** (whether/when native HarmonyOS and Aurora OS clients exist to consume the contract-only token bindings). The AR/QT shim halves are `[BUILD-NEW]` in `helix_shims` (or a sibling), owned by the VPN/client program, not by the design library.

- Registry updates in index.md are intentionally **not** made by this file (atomic scope: this file records its own findings and the §6 ruling; the registry pass mirrors them).

6. SwiftUI decision memo (THREADY-DES-LIB-02)

`[OPERATOR — RULING RECORDED, 2026-07-22]` **— Option B adopted.** The operator ruling adopts **Option B: Compose-Multiplatform-only with a thin SwiftUI host shell**, per this memo's recommendation and the platform-map §2.1 sanctioned path ("SwiftUI / iOS via Compose Multiplatform (thin shims ASSUMED)"). iOS idiom deviations are accepted as documented in Option B's risks below (e.g. Compose shimmer instead of `.redacted` skeletons). The remaining engineering dependency is the `UI-Components-KMP` Yole→Thready retheme, which consumes the committed tokens-bridge `ThreadyColors.kt` contract. `THREADY-DES-LIB-02` is **CLOSED (decided)** — see §5. The memo below is preserved as the decision record.

Context: platform-map §2.1 shows the sanctioned iOS path as "SwiftUI / iOS via Compose Multiplatform (thin shims ASSUMED)" — "the sanctioned iOS path is Compose Multiplatform (or thin SwiftUI shims), not a" dedicated SwiftUI component package, and `THREADY-DES-LIB-02` records that **no in-house SwiftUI package exists or is named in the decision matrix** `[VERIFIED — platform-map §2.1/§6; index.md registry]`.

Option A — dedicated SwiftUI component package (`[BUILD-NEW]`)

Evidence-based assessment:

- **Nothing to start from.** No in-house SwiftUI component code exists in any inspected repo (`design_system` = Angular + tokens; `UI-Components-React` = React; `UI-Components-KMP` = Compose; `helix_shims` apple/ = a NetworkExtension spike with zero UI) `[VERIFIED — this file §2–§3; platform-map §2]`. The entire ~20-component × variant × state matrix would be new code, plus a new visual-regression surface.
- **Token bridge is ready, so the *styling* half is cheap:** the generated `ThreadyTokens.swift` contract (Light/Dark colors, spacing, radius, type scale) already exists — but it is explicitly "contract only, in case SwiftUI shims materialize" `[VERIFIED — tokens-bridge/README.md]`.
- **Upside:** fully native look/feel and idioms (e.g. `.redacted(reason: .placeholder)` skeletons the matrix already assigns to SwiftUI, platform-map §3 notes); no Compose-on-iOS runtime.
- **Cost:** duplicates the component library on a platform with no current product surface; contradicts the standing decision matrix without an operator override.

Option B — Compose-Multiplatform-only, with a thin SwiftUI host shell

Evidence-based assessment:

- **Reuses code that exists:** `UI-Components-KMP` ships real Compose components at HEAD; its known defect is branding, not structure — `Theme.kt` is "branded for another product (Yole, Material-red)"

`[VERIFIED — platform-map §2 rev 2]`. The remediation is a token retheme, the direct analogue of the React re-bind executed under react-token-rebind.md — a bounded, contract-driven fix rather than a new library.

- **In-house precedent for the host-shell pattern:** `helix_shims` `apple/ios-spike/PacketTunnelProvider.swift` already demonstrates a thin Swift host over a shared non-Swift core (§2.3) — the same shape a SwiftUI app shell hosting Compose UI takes `[VERIFIED — this file §2.3]`.
- **Single implementation** of every component/state/theme; the theme×state audit and visual-regression bank stay one-platform-per-component instead of forking.
- **Risks (honest):** Compose Multiplatform iOS runtime maturity/binary size; deviations from iOS idioms where the matrix expects native behavior (e.g. `.redacted` skeletons would be Compose shimmer instead); a thin SwiftUI shell is still needed for app chrome, so "zero Swift UI code" is not on the table either way.

Recommendation (non-binding)

On the evidence, **Option B** is the lower-cost, decision-matrix-consistent path: the only verified in-house mobile component asset is Compose (`UI-Components-KMP`), its blocking defect is a retheme with a proven remediation pattern, and the SwiftUI token contract stays on the shelf ready if a genuine native-SwiftUI surface requirement ever materializes. Option A should only be taken with an explicit operator override of the decision matrix and a named product surface that demands native SwiftUI.

Decision owner: operator. Status: DECIDED — Option B adopted (operator ruling, 2026-07-22).

Made with love ♥ by Helix Development.